A MINI PROJECT REPORT

NetSleuth: An Explainable Network Attack Timeline Reconstruction and Threat Analysis System for Cloud Flow Logs

Submitted for the course

Cybersecurity and Cloud Computing

Course Code: [COURSE CODE]

Submitted by

[STUDENT NAME]

Roll / Enrolment No.: [ROLL NUMBER]

Under the guidance of

[GUIDE NAME AND DESIGNATION]

[DEPARTMENT NAME]

[COLLEGE / UNIVERSITY NAME]

Academic Year: [YEAR] | October 2026

Certificate

This is to certify that the mini project report entitled **"NetSleuth: An Explainable Network Attack Timeline Reconstruction and Threat Analysis System for Cloud Flow Logs"** is a bona fide record of the work carried out by [STUDENT NAME] (Roll No. [ROLL NUMBER]) in partial fulfilment of the requirements of the course **Cybersecurity and Cloud Computing** at [COLLEGE / UNIVERSITY NAME] during the academic year [YEAR].

The work presented in this report has not been submitted elsewhere for any other degree, diploma or course.

_____	_____
[GUIDE NAME]	[HEAD OF DEPARTMENT]
Project Guide	Head of Department

Acknowledgement

[Replace this paragraph with your own words.] I would like to thank my project guide, [GUIDE NAME], for the guidance and feedback given during this project. I also thank the faculty of the [DEPARTMENT NAME] for the course content that made this work possible, and my family and friends for their support.

Abstract

Cloud platforms record large amounts of network flow data, such as AWS VPC Flow Logs, but reading these records by hand is slow, and the alerts raised from them are often scattered and unexplained. This project presents NetSleuth, a small rule-based tool that analyses cloud-style flow logs and presents the results as clear, explainable incidents.

The system applies three sliding-window rules to detect port scans, host scans and data exfiltration. Alerts from the same source IP that occur close together in time are grouped into one incident, mapped to MITRE ATT&CK techniques, and given a risk score from 0 to 100 in which every point can be traced. A timeline, a host communication graph and a Streamlit dashboard help the analyst follow the attack step by step.

The system was tested on five synthetic scenarios generated with a fixed random seed. It detected all attack flows with no false alarms on these datasets. Because the data is self-generated and the thresholds are assumed values, these results show that the logic works as designed. They do not prove that it will perform equally well on real networks.

Keywords: network flow logs, AWS VPC Flow Logs, port scan, data exfiltration, incident correlation, explainable risk scoring, MITRE ATT&CK, Streamlit.

Table of Contents

1. Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope 2
1.5 Organisation of the Report 2
2. Background and Cloud Relevance 3
2.1 Network Flow Logs 3
2.2 Relevance to Cloud Security 3
2.3 Suspicious Behaviours Studied 3
2.4 MITRE ATT&CK and Attack Stages 3
2.5 Why Explainable Rules 3
3. System Design 5
3.1 Overview 5
3.2 Modules 5
3.3 Input Data and Cloud Connection 5
3.4 Data Validation 6
3.5 Synthetic Datasets 6
4. Detection, Correlation and Scoring 7
4.1 Detection Rules 7
4.2 Alert Contents 7
4.3 MITRE ATT&CK Mapping 7
4.4 Incident Grouping 7
4.5 Explainable Risk Scoring 8
4.6 Worked Example 8
4.7 Response Advice 9
4.8 Separation of Labels from Detection 9
5. Dashboard 10
5.1 Overview 10
5.2 Screenshots 11
6. Testing and Results 13
6.1 Testing Approach 13
6.2 Evaluation Method 13
6.3 Detection Results 13
6.4 Incident Results 14
6.5 Observations 14
6.6 How to Read These Results 14
7. Limitations 15
8. Future Work 15
9. Conclusion 15
References 16
Appendix A: Setup and Running 17
Appendix B: Parameter Summary 17

1. Introduction

1.1 Background

Cloud platforms record information about network traffic so that customers can monitor and secure their systems. In Amazon Web Services (AWS), VPC Flow Logs capture information about the IP traffic going to and from network interfaces in a virtual private cloud. Each record summarises one network flow, for example who connected to whom, on which port, how many bytes were sent, and whether the connection was accepted or rejected.

These logs are very useful for security work, but even a small cloud deployment can produce a very large number of records. A person cannot read them line by line, and simple tools that raise one alert per suspicious event leave the analyst with a long list of unconnected warnings.

1.2 Problem Statement

Security teams that work with flow logs face four connected problems:

1. **Too many records and scattered alerts.** Alerts raised for single events are not linked to each other, which leads to alert fatigue. Important events can be missed among low-priority noise.
2. **Related events look unrelated.** A typical intrusion has stages: looking for open services, looking for other machines to reach, and finally taking data out. Tools often show these as three separate warnings instead of one story.
3. **Unexplained scores.** Some tools use machine learning to produce a risk score without giving the reason. An engineer cannot safely act on a score that cannot be explained.
4. **Tools that are hard to study.** Many production systems need heavy infrastructure and configuration, which makes them difficult for students to learn from.

NetSleuth addresses these problems with a small, transparent pipeline. It reads cloud-style flow records, applies three rule-based detectors, groups related alerts into incidents, scores each incident with a formula that can be checked by hand, and shows the result as a timeline, a network graph and a dashboard.

1.3 Objectives

1. Detect three suspicious behaviours (port scan, host scan and data exfiltration) using clear, configurable rules.
2. Group related alerts from the same source into one incident, using a time-gap rule.
3. Give every incident a risk score from 0 to 100 with a line-by-line explanation.
4. Rebuild the order of events as a timeline and show host communication as a graph.
5. Measure detection quality against labelled synthetic data using precision, recall and F1 score.
6. Present everything in an interactive dashboard that works on a single laptop.

1.4 Scope

In scope:

- Analysis of flow-log CSV files in a simplified AWS VPC Flow Log format.
- Three rule-based detectors, incident grouping, scoring, timeline and graph.
- Evaluation on five synthetic scenarios and an interactive dashboard.

Out of scope:

- Live connection to AWS or any other cloud account (listed as future work).
- Inspection of packet contents, machine learning models and real-time streaming.
- Automatic blocking or any change to firewall or cloud settings. The tool only advises.

1.5 Organisation of the Report

Chapter 2 gives the background on flow logs, cloud security and the attacks studied. Chapter 3 describes the system design and data. Chapter 4 explains the detection rules, incident grouping and scoring. Chapter 5 presents the dashboard. Chapter 6 reports testing and results. Chapters 7 to 9 cover limitations, future work and the conclusion.

2. Background and Cloud Relevance

2.1 Network Flow Logs

A flow log is a summary of network connections. It does not hold the data that was sent. It records details about the connection, such as the source and destination address, the destination port, the protocol, the number of bytes, the time, and whether the connection was allowed. Because no message content is stored, flow logs cannot show what was inside the traffic.

2.2 Relevance to Cloud Security

Cloud providers follow a shared responsibility model. The provider secures the underlying infrastructure, while the customer is responsible for securing and monitoring their own workloads, network rules and data. Network flow logs are one of the main tools a customer has to watch what their cloud resources are doing.

In AWS, VPC Flow Logs can be published to Amazon CloudWatch Logs or Amazon S3. NetSleuth reads a simplified version of the same fields, so the same logic could be applied to real exported logs.

2.3 Suspicious Behaviours Studied

Table 2.1: *Behaviours detected by NetSleuth*

Behaviour	Plain meaning	How it appears in flow logs
Port scan	An attacker checks which services a single machine offers, like trying many doors on one building.	One source IP connects to many different ports on one destination IP in a short time.
Host scan	An attacker looks for other reachable machines, like trying the same door on many buildings.	One source IP connects to many different destination IPs on the same port in a short time.
Data exfiltration	Data is taken out of the network to an outside address.	A large number of bytes leaves an internal host for an external IP within a short time.

2.4 MITRE ATT&CK and Attack Stages

MITRE ATT&CK is a public knowledge base that describes the tactics and techniques attackers use. NetSleuth links each detection rule to one ATT&CK technique so that an alert is described in a standard language. The report also uses attack-stage names (reconnaissance, lateral movement preparation and actions on objectives) to show where each alert fits in an intrusion.

2.5 Why Explainable Rules

Machine learning can find unusual patterns without fixed rules, but it needs training data and its decisions are often hard to explain. Rules are simpler. They are easy to read, easy to test, need no training data, and every alert can be explained in one sentence. Their weakness is that

thresholds must be chosen and tuned, and they only find the patterns they were written for. This project uses rules because the goal is transparency and learning.

3. System Design

3.1 Overview

NetSleuth is a one-way pipeline. A flow-log CSV is cleaned and checked, passed through three detectors, grouped into incidents and scored. The results feed a Streamlit dashboard. A separate evaluation module compares alerts with labels from the synthetic data.

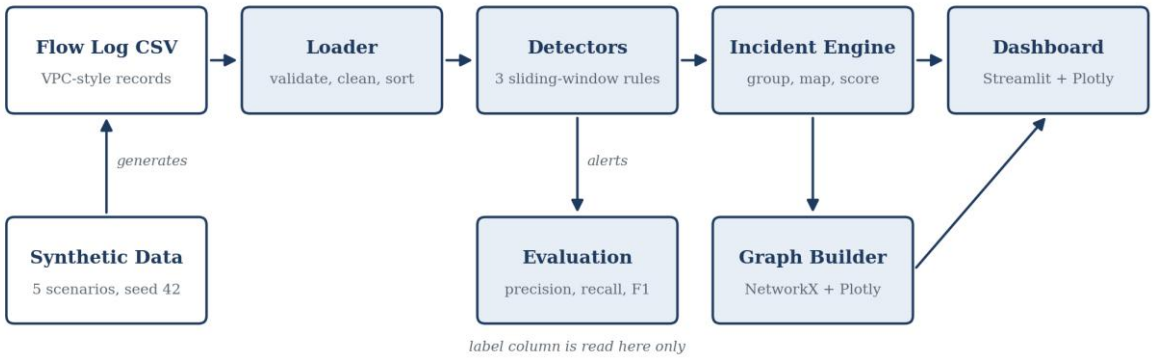


Figure 3.1: *NetSleuth system architecture*

3.2 Modules

Table 3.1: *Main modules and their roles*

Module	File	Role
Loader	`loader.py`	Reads the CSV, checks required columns, removes invalid rows, normalises timestamps to UTC and sorts by time.
Detectors	`detectors.py`	Runs the three sliding-window rules and returns alerts with a plain-English reason and the contributing flow rows.
Incident engine	`incidents.py`	Groups alerts into incidents, maps them to MITRE ATT&CK, calculates the risk score and builds the timeline and response advice.
Graph builder	`graph.py`	Builds a directed host-to-host graph with NetworkX and draws it with Plotly.
Evaluation	`evaluation.py`	Compares detected flows with ground-truth labels and computes TP, FP, FN, precision, recall and F1.
Synthetic data	`synthetic.py`	Generates the five test scenarios with a fixed random seed.
Dashboard	`app.py`	Streamlit interface that ties all modules together.

All thresholds and weights are kept in one configuration file, config.py (located in src/netsleuth/config.py), so that no unexplained numbers appear in the code.

3.3 Input Data and Cloud Connection

NetSleuth accepts a simplified subset of the AWS VPC Flow Log fields. Table 3.2 shows how each column relates to the AWS field.

Table 3.2: *Mapping of NetSleuth columns to AWS VPC Flow Log fields*

NetSleuth column	AWS VPC Flow Log field	Meaning
`timestamp`	`start` / `end`	Event time, converted to ISO-8601 UTC.
`src_ip`	`srcaddr`	Source IPv4 address.
`dst_ip`	`dstaddr`	Destination IPv4 address.
`dst_port`	`dstport`	Destination port (1 to 65,535).
`protocol`	`protocol`	Transport protocol, TCP or UDP.
`bytes`	`bytes`	Data volume of the flow.
`action`	`action`	ACCEPT or REJECT.
`direction`	Derived	INBOUND, OUTBOUND or INTERNAL, worked out from the addresses.
`label` (optional)	Not an AWS field	Ground-truth tag used only by the evaluation module.

In a real AWS setup, flow logs would be delivered to Amazon S3 or CloudWatch Logs and could be exported to this CSV layout. Reading logs directly from AWS is not implemented and is listed under future work.

3.4 Data Validation

The loader rejects or removes bad data before any detection happens. It:

- checks that all required columns are present;
- removes rows with an invalid IPv4 address, a port outside 1 to 65,535 or a negative byte count, and reports how many were removed;
- converts timestamps to UTC and sorts the flows in time order.

3.5 Synthetic Datasets

Real flow logs contain private information and do not come with labels that say which flows are attacks. For this reason the project uses synthetic data made by `synthetic.py` with a fixed random seed (42), so every run gives the same files. Each of the five scenarios holds between 224 and 320 flows. Normal background traffic (web browsing, DNS queries, SSH administration and email) is mixed with a controlled attack pattern. One scenario contains normal traffic only. The files are `scenario1_port_scan.csv`, `scenario2_host_scan.csv`, `scenario3_data_exfiltration.csv`, `scenario4_multi_stage.csv` and `scenario5_benign.csv`.

4. Detection, Correlation and Scoring

4.1 Detection Rules

Each rule looks at a moving time window over the sorted flows. When the condition is met, an alert is raised. Table 4.1 lists the three rules and the reason behind each threshold.

Table 4.1: *Detection rules and thresholds*

Rule	Window	Condition	Severity	Reason for threshold
Port scan	60 s	One source IP reaches 10 or more different destination ports on one destination IP.	MEDIUM	Normal clients use one or two service ports. Ten ports in a minute suggests automated probing.
Host scan	60 s	One source IP reaches 5 or more different destination IPs on the same port.	MEDIUM	A workstation talks to a few known servers. Five peers on one port in a minute suggests a sweep.
Data exfiltration	120 s	Outbound traffic from one internal IP to an external IP totals 500,000 bytes (about 500 KB) or more.	HIGH	Ordinary outbound requests are small. A bulk upload in two minutes needs review.

These thresholds are assumed values chosen for demonstration. They are not taken from measurements of real networks (see Chapter 7).

4.2 Alert Contents

Every alert records the rule name, severity, source IP, target IPs or ports, start and end time, a plain-English reason sentence and the row numbers of the flows that caused it. An example reason sentence is: “10.0.0.5 tried 14 different ports on 10.0.1.20 in 38 seconds.”

4.3 MITRE ATT&CK Mapping

Table 4.2: *Mapping of rules to MITRE ATT&CK and attack stage*

Rule	Technique	Tactic	Attack stage
Port scan	T1046 Network Service Discovery	Discovery (TA0007)	Reconnaissance
Host scan	T1018 Remote System Discovery	Discovery (TA0007)	Lateral movement preparation
Data exfiltration	T1048 Exfiltration Over Alternative Protocol	Exfiltration (TA0010)	Actions on objectives (data theft)

4.4 Incident Grouping

Alerts are grouped into incidents using two conditions:

- **Same source.** Alerts must come from the same source IP.

- **Close in time.** If the gap between the end of one alert and the start of the next is 300 seconds (5 minutes) or less, the alerts belong to the same incident. A larger gap starts a new incident.

The 5-minute gap is an assumed session boundary. The idea is that a person working through scan, spread and theft in one sitting will do so within minutes.

4.5 Explainable Risk Scoring

The risk score of an incident is a simple sum, limited to 100:

Risk Score = min(100, Base Points + Extra Alert Points + Volume Points)

Table 4.3: *Score components*

Component	Rule	Points
Base points	Highest severity among the incident's alerts: LOW, MEDIUM, HIGH	10, 30, 50
Extra alert points	10 points for each alert after the first, 10 x (N - 1)	10 per extra alert
Volume points	Total exfiltrated bytes of 500,000 or more / 5,000,000 or more / below 500,000	10 / 20 / 0

Table 4.4: *Priority levels*

Score	Level
0 to 29	Low
30 to 59	Medium
60 to 79	High
80 to 100	Critical

4.6 Worked Example

Take one internal host, 10.0.1.99, that raises three alerts within 5 minutes: a port scan (MEDIUM), a host scan (MEDIUM) and an exfiltration of 6,300,000 bytes (HIGH). The score is calculated as follows.

1. Base points: the highest severity is HIGH, so **+50**.
2. Extra alerts: 3 - 1 = 2 extra alerts, so 2 x 10 = **+20**.
3. Volume points: 6.3 MB is above 5 MB, so **+20**.
4. Total: 50 + 20 + 20 = 90. The cap of 100 does not apply, so the score is **90 (Critical)**.

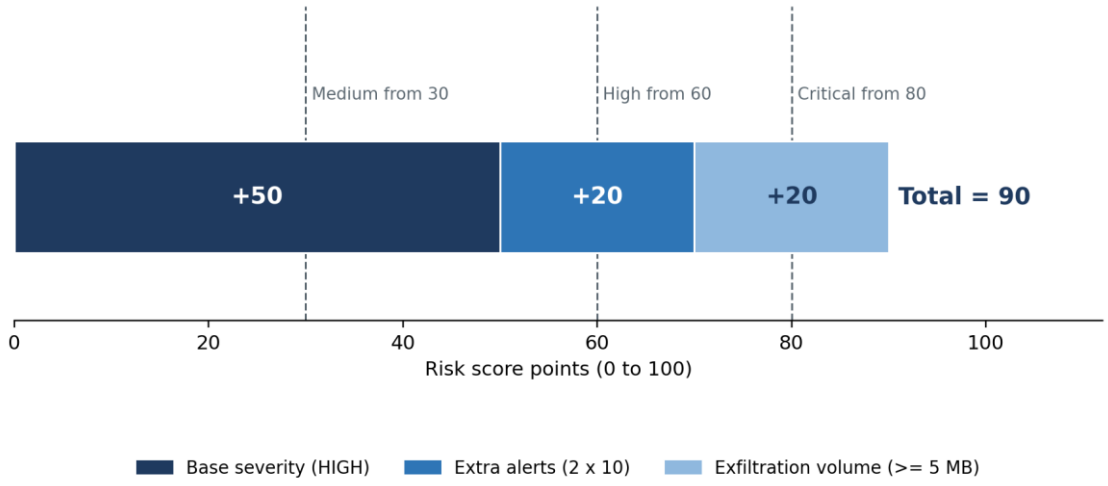


Figure 4.1: *Score breakdown for the worked example*

This example matches the Scenario 4 result of 90 (Critical) in Chapter 6, where 6,300,000 bytes (6.3 MB) were transferred.

4.7 Response Advice

For each incident the dashboard shows suggested response steps, for example tightening or revoking the related security group rules, isolating the affected host, or adding a host firewall block. The advice is only text. NetSleuth never changes any firewall or cloud setting.

4.8 Separation of Labels from Detection

The optional `label` column is read only by the evaluation module. The detectors work on addresses, ports, bytes and time only, so they cannot use the answer key. A test confirms that detection gives the same result when the label column is removed.

5. Dashboard

5.1 Overview

The dashboard is built with Streamlit and Plotly and runs locally in a web browser. The user selects one of the five synthetic scenarios, or uploads a CSV file, and the whole pipeline runs on the chosen data. A dark theme is used, and hosts in the graph are coloured red for attackers, amber for victims and cyan for other hosts.

Table 5.1: *Dashboard views*

View	What it shows
Threat Incidents	Incident cards with priority level, risk score, deterministic line-by-line score breakdown, MITRE ATT&CK progression, targeted endpoints, containment playbooks, and expandable alert milestones.
Attack Timeline	Interactive chronological scatter plot of incidents by kill chain stage and a detailed chronological incident audit trail table.
Network Topology	Directed host communication graph with tactical SOC colour coding (red attacker, amber victims, cyan normal hosts) and top communicating host pairs table.
Flow Telemetry	Donut charts for protocol and firewall disposition, bar chart for top targeted ports, and interactive flow explorer with IP search and direction/action filters.
Forensic Dossier & Evaluation	One-click export and live preview of the formal markdown forensic investigation dossier, alongside the scientific ground-truth evaluation table across all benchmark scenarios.

5.2 Screenshots

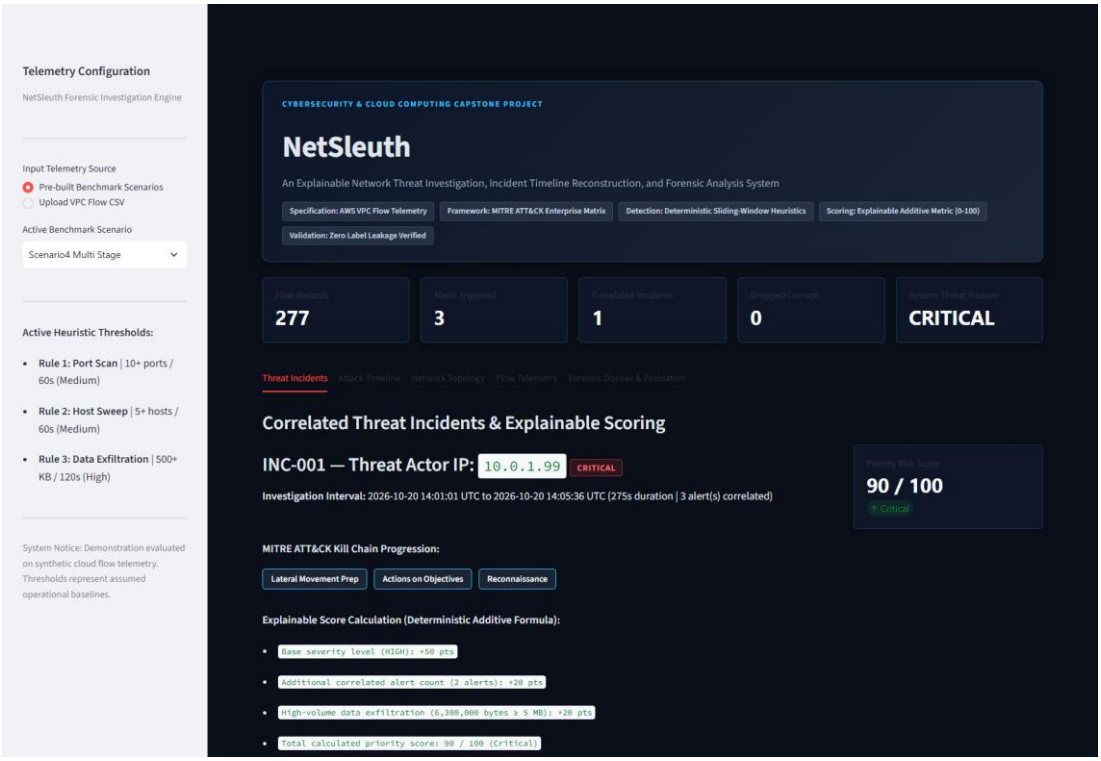


Figure 5.1: *Incidents and Score view for the multi-stage scenario*

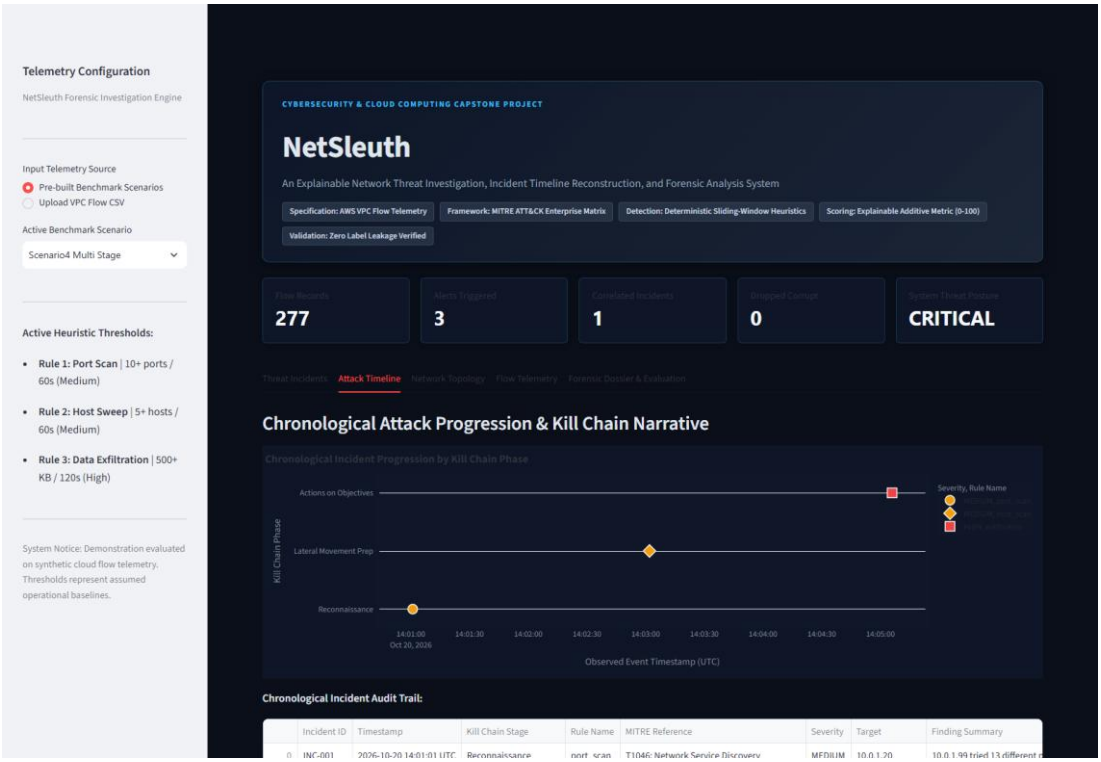


Figure 5.2: *Timeline view of the multi-stage attack*

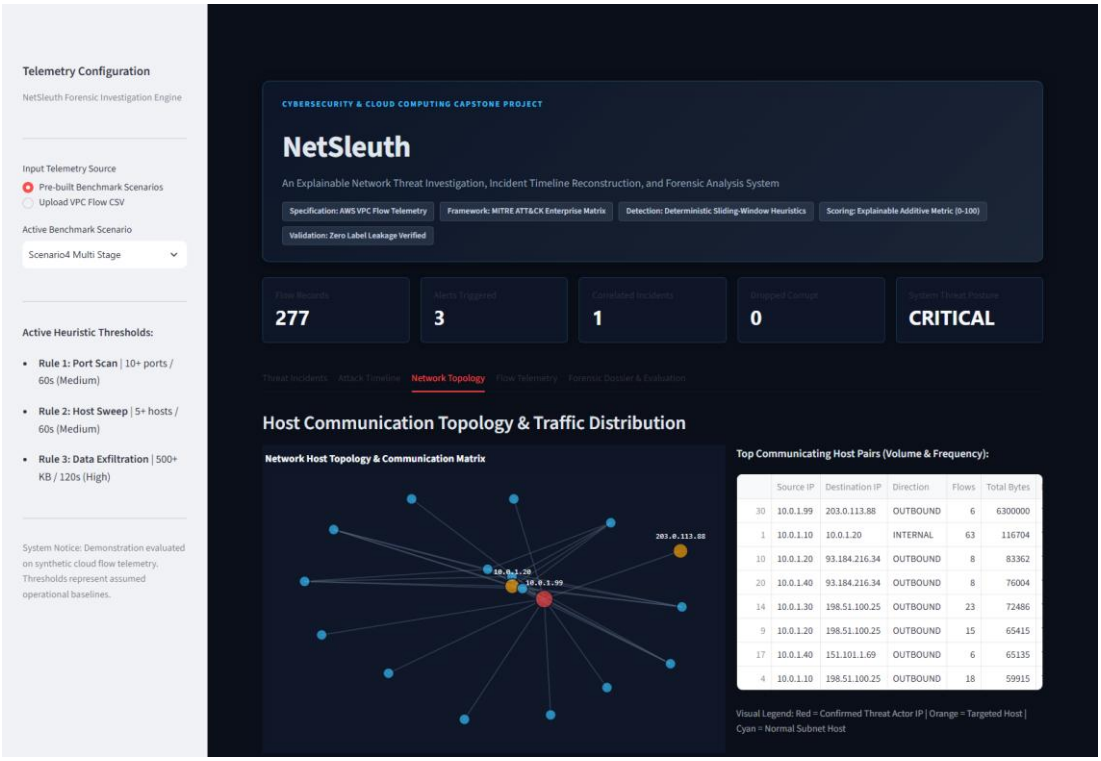


Figure 5.3: *Host communication graph (red: attacker, amber: victims, cyan: other hosts)*

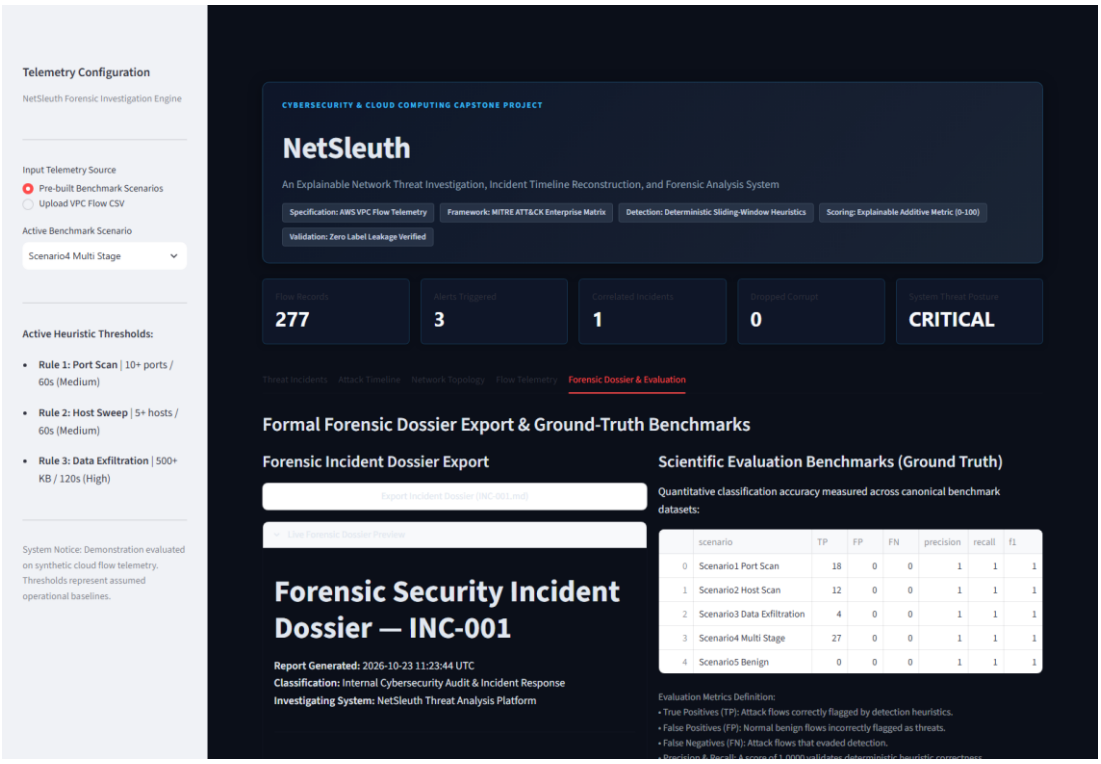


Figure 5.4: *Evaluation results for all scenarios*

6. Testing and Results

6.1 Testing Approach

Automated tests were written with pytest. They cover the main logic of the project:

- each detector fires when the threshold is just met and does not fire one step below it;
- the benign scenario produces no alerts;
- alerts less than 5 minutes apart form one incident and alerts further apart form two;
- a score calculated by hand matches the program, and the score never exceeds 100;
- the loader removes invalid rows;
- detection does not change when the label column is removed.
- MITRE ATT&CK technique mapping and markdown forensic dossier generation produce valid outputs.

All 10 automated unit tests pass.

6.2 Evaluation Method

Detection is measured at flow level. A flow counts as detected if it appears in the contributing flows of any alert.

- **True positive (TP):** an attack flow that was detected.
- **False positive (FP):** a normal flow that was detected.
- **False negative (FN):** an attack flow that was missed.

$$Precision = TP / (TP + FP) \qquad Recall = TP / (TP + FN) \qquad F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

In Scenario 5 there are no attack flows, so precision and recall have no meaning. The tool shows 1.0000 for this scenario by convention. The useful result for Scenario 5 is that FP = 0.

6.3 Detection Results

Table 6.1: *Flow-level evaluation on the five synthetic scenarios*

Dataset	Threat profile	Flows	TP	FP	FN	Precision	Recall	F1
1. Port scan	External port probe on one host, plus normal traffic	238	18	0	0	1.0000	1.0000	1.0000
2. Host scan	Internal SMB subnet sweep, plus normal traffic	232	12	0	0	1.0000	1.0000	1.0000
3. Data exfiltration	Large outbound transfer, plus normal traffic	224	4	0	0	1.0000	1.0000	1.0000
4. Multi-stage attack	Port scan, host sweep, then exfiltration	277	27	0	0	1.0000	1.0000	1.0000
5. Benign baseline	Normal traffic only	320	0	0	0	1.0000*	1.0000*	1.0000*

**Defined by convention because the scenario has no attack flows.*

6.4 Incident Results

In Scenario 4, the three alerts (port scan, host scan and exfiltration) were grouped into one incident, INC-001, with a score of 90 (Critical). This shows the main purpose of the system: three separate warnings become one connected story.

Table 6.2: *Alerts, incidents and scores per scenario*

Scenario	Alerts	Incidents	Score	Level
Scenario 1	1	1 (INC-001)	30	Medium
Scenario 2	1	1 (INC-001)	30	Medium
Scenario 3	1	1 (INC-001)	60	High
Scenario 4	3	1 (INC-001)	90	Critical
Scenario 5	0	0	None	None

6.5 Observations

- In Scenario 5, 320 normal flows produced no alerts, so these thresholds did not raise false alarms on this data.
- In Scenarios 1 to 4, every attack flow was found (FN = 0).
- In Scenario 4, grouping and scoring turned three alerts into one incident with a clear explanation.

6.6 How to Read These Results

The datasets were generated by the same project that tests them, and the thresholds were chosen with the same attack patterns in mind. Perfect scores are therefore expected. They show that the rules, the grouping and the scoring work as designed. They do not show how well the system would perform on real cloud traffic, where backups, monitoring tools and busy periods can look like attacks, and where attackers may avoid the fixed thresholds.

7. Limitations

- **Synthetic data.** All results come from self-generated data with a fixed seed. Real networks are noisier.
- **Assumed thresholds.** The values 10 ports, 5 hosts, 60 and 120 seconds, and 500 KB are demonstration values. A real deployment would need to tune them to its own traffic.
- **Easy to avoid.** A scan spread over a period longer than the window, or a slow upload kept below the byte threshold, would not be detected by fixed rules.
- **Metadata only.** Flow logs do not contain packet contents, so attacks hidden inside application data cannot be seen.
- **Batch processing.** The tool reads a finished CSV file. It does not process a live stream.
- **Possible false alarms.** Approved scanners, backup jobs and bulk transfers can match the rules. There is no whitelist.
- **Flow-level evaluation.** The evaluation counts flows, not incidents, and uses labels created by the same project.

8. Future Work

1. **Live cloud input.** Read flow logs directly from Amazon S3 or CloudWatch Logs using the AWS SDK.
2. **Tuning on real logs.** Replace fixed thresholds with values based on each network's normal behaviour, for example rolling averages and standard deviations.
3. **Whitelist.** Let the user exempt approved scanners and backup services from alerts.
4. **Comparison with anomaly detection.** Compare the rule-based approach with a simple statistical or machine learning method.
5. **Controlled response.** Connect the response advice to approval-based automation, for example updating a security group only after an analyst confirms.

9. Conclusion

This project built NetSleuth, a small tool that turns cloud-style flow logs into explained incidents. Three simple rules detect port scans, host scans and data exfiltration. Alerts from the same source are grouped, mapped to MITRE ATT&CK, scored with a formula that can be checked by hand, and shown on a timeline and a graph in a dashboard.

On five synthetic scenarios the system found every attack flow and raised no false alarms. Because the data is self-generated and the thresholds are assumed, the main value of the project is the clear, explainable design and the working pipeline, not a claim of real-world accuracy. The project gave practical experience with cloud flow logs, basic attack detection, incident correlation and testing, and it provides a base for the future work described in Chapter 8.

References

1. Amazon Web Services, “Logging IP traffic using VPC Flow Logs,” Amazon VPC User Guide. docs.aws.amazon.com
2. National Institute of Standards and Technology, “Guide to Integrating Forensic Techniques into Incident Response,” NIST Special Publication 800-86. nist.gov
3. The MITRE Corporation, “MITRE ATT&CK: Enterprise Matrix.” attack.mitre.org
4. NetworkX Developers, “NetworkX Documentation.” networkx.org
5. Streamlit Inc., “Streamlit Documentation.” docs.streamlit.io
6. Plotly Technologies Inc., “Plotly Python Graphing Library.” plotly.com/python
7. The pandas Development Team, “pandas Documentation.” pandas.pydata.org

Appendix A: Setup and Running

Requirements: Python 3.10 or newer and Git.

```
git clone https://github.com/NileshSethi/NetSleuth.git
cd NetSleuth
python -m venv .venv
.venv\Scripts\Activate.ps1        (Windows PowerShell)
source .venv/bin/activate          (macOS / Linux)
pip install -r requirements.txt
pytest                              (run the tests)
streamlit run app.py                (open http://localhost:8501)
```

Appendix B: Parameter Summary

Table B.1: *Thresholds and weights used by the system*

Parameter	Value
Port scan	10 or more distinct ports on one target within 60 s
Host scan	5 or more distinct targets on one port within 60 s
Exfiltration	500,000 bytes or more, internal to external, within 120 s
Incident gap	300 s between alerts from the same source
Base points	LOW 10, MEDIUM 30, HIGH 50
Extra alert points	10 for each alert after the first
Volume points	10 at 500,000 bytes or more, 20 at 5,000,000 bytes or more
Score cap	100
Levels	Low below 30, Medium 30 to 59, High 60 to 79, Critical 80 and above